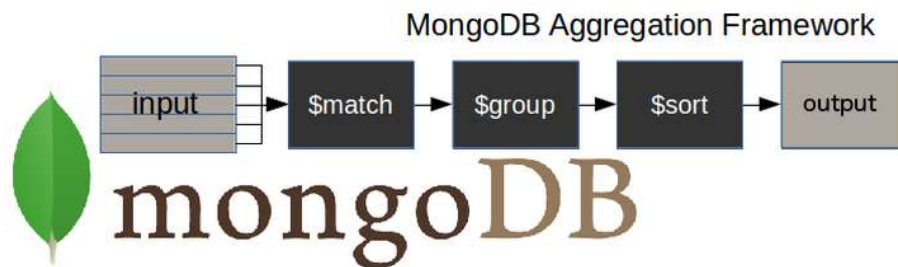


Investigación- Agregaciones en mongo

La **agregación** es una forma de procesar un gran número de documentos de una colección haciéndolos pasar por distintas etapas. Las etapas constituyen lo que se conoce como "pipeline". Las etapas de un pipeline pueden filtrar, ordenar, agrupar, remodelar y modificar los documentos que pasan por el pipeline.

Uno de los usos más comunes de la agregación es calcular valores agregados para grupos de documentos. Esto es similar a la agregación básica disponible en SQL con la cláusula GROUP BY y las funciones COUNT, SUM y AVG. Sin embargo, MongoDB Aggregation va más allá y también puede realizar uniones de tipo relacional, remodelar documentos, crear nuevas colecciones y actualizar las existentes, etc.

¿Cómo funciona el proceso de agregación de MongoDB?



- **\$match** etapa - filtra los documentos con los que necesitamos trabajar, los que se ajustan a nuestras necesidades
- **\$group** etapa - realiza el trabajo de agregación
- **\$sort** etapa - ordena los documentos resultantes de la forma que deseemos (ascendente o descendente)

La entrada de la cadena puede ser una sola colección, en la que se pueden fusionar otras más adelante.

A continuación, la tubería realiza transformaciones sucesivas en los datos hasta alcanzar nuestro objetivo.

De este modo, podemos dividir una consulta compleja en etapas más sencillas, en cada una de las cuales completamos una operación diferente sobre los datos. Así, al final de la cadena de consultas, habremos conseguido todo lo que queríamos.

Ejemplos de agregados en MongoDB

MongoDB \$match

En `$match` etapa nos permite elegir sólo aquellos documentos de una colección con los que queremos trabajar. Para ello, filtra los que no cumplen nuestros requisitos.

En el siguiente ejemplo, sólo queremos trabajar con aquellos documentos en los que se especifique que `Spain` es el valor del campo `country` y `Salamanca` es el valor del campo `city`.

Para obtener una salida legible, voy a añadir `.pretty()` al final de todos los comandos.

```
db.universities.aggregate([  
  
  { $match : { country : 'Spain', city : 'Salamanca' } }  
  
]).pretty()
```

MongoDB \$project

Es raro que necesite recuperar todos los campos de sus documentos. Es una buena práctica devolver sólo los campos que necesita para evitar procesar más datos de los necesarios.

En `$project` etapa se utiliza para hacer esto y para añadir cualquier campo calculado que necesite.

En este ejemplo, sólo necesitamos los campos `country`, `city` y `name`.

En el código que sigue, ten en cuenta que:

- Debemos escribir explícitamente `_id : 0` cuando este campo no es obligatorio
- Aparte del `_id` basta con especificar únicamente los campos que necesitamos obtener como resultado de la consulta

Este etapa ...

```
db.universities.aggregate([  
  
  { $project : { _id : 0, country : 1, city : 1, name : 1 } }  
  
]).pretty()
```

MongoDB \$group

Con el \$group etapapodemos realizar todas las consultas de agregación o resumen que necesitemos, como encontrar recuentos, totales, medias o máximos.

En este ejemplo, queremos saber el número de documentos por universidad en nuestro 'universitiescolección:

La consulta ...

```
db.universities.aggregate([

{ $group : { _id : '$name', totaldocs : { $sum : 1 } } }

]).pretty()
```

Operadores de agregación \$group de MongoDB

\$group etapa admite determinadas expresiones (operadores) que permiten a los usuarios realizar operaciones aritméticas, array, booleanas y de otro tipo como parte de la cadena de agregación.

Operador	Significado
\$count	Calcula la cantidad de documentos del grupo dado.
\$max	Muestra el valor máximo del campo de un documento en la colección.
\$min	Muestra el valor mínimo de un campo del documento en la colección.
\$avg	Muestra el valor medio del campo de un documento de la colección.
\$suma	Suma los valores especificados de todos los documentos de la colección.
empujar	Añade valores adicionales en la dirección array del documento resultante.

MongoDB \$out

Se trata de un tipo de etapa poco habitual, ya que permite trasladar los resultados de la agregación a una nueva colección, o a una ya existente tras eliminarla, o incluso añadirlos a los documentos existentes (novedad en la versión 4.1.2).

En `$out` etapa debe ser el último etapa de la cadena.

Por primera vez, utilizamos una agregación con más de un etapa. Ahora tenemos dos, a `$group` y un `$out`:

```
db.universities.aggregate([  
  
  { $group : { _id : '$name', totaldocs : { $sum : 1 } } },  
  
  { $out : 'aggResults' }  
  
])
```

MongoDB \$unwind

En `$unwind` etapa en MongoDB se encuentra comúnmente en un pipeline porque es un medio para un fin.

No se puede trabajar directamente sobre los elementos de un array dentro de un documento con etapas como `$group`. En `$unwind` nos permite trabajar con los valores de los campos dentro de un array.

Si en los documentos de entrada hay un campo array , a veces tendrá que imprimir el documento varias veces, una por cada elemento de array.

En cada copia del documento se sustituye el campo array por el elemento sucesivo.

MongoDB \$sort

Necesita el `$sort` etapa para ordenar los resultados por el valor de un campo específico.

Por ejemplo, ordenemos los documentos obtenidos como resultado del `$unwind` etapa por número de alumnos en orden descendente.

Para obtener un resultado menor, voy a proyectar sólo el año y el número de alumnos.

```
db.universities.aggregate([
```

```
{ $match : { name : 'USAL' } },

{ $unwind : '$students' },

{ $project : { _id : 0, 'students.year' : 1, 'students.number' : 1 } },

{ $sort : { 'students.number' : -1 } }

]).pretty()
```

MongoDB \$limit

¿Y si sólo le interesan los dos primeros resultados de su consulta? Es tan sencillo como

```
db.universities.aggregate([

  { $match : { name : 'USAL' } },

  { $unwind : '$students' },

  { $project : { _id : 0, 'students.year' : 1, 'students.number' : 1 } },

  { $sort : { 'students.number' : -1 } },

  { $limit : 2 }

]).pretty()

{ "students" : { "year" : 2014, "number" : 24774 } }

{ "students" : { "year" : 2015, "number" : 23166 } }
```

\$addFields

Es posible que necesite realizar algunos cambios en su salida en forma de nuevos campos. En el siguiente ejemplo, queremos añadir el año de fundación de la universidad.

```
db.universities.aggregate([

  { $match : { name : 'USAL' } },

  { $addFields : { foundation_year : 1218 } }
```

```
]).pretty()
```

MongoDB \$lookup

Dado que MongoDB se basa en documentos, podemos darles la forma que necesitemos. Sin embargo, a menudo es necesario utilizar información de más de una colección.

Utilización de \$lookup A continuación se muestra una consulta agregada que combina campos de dos colecciones.

```
db.universities.aggregate([

  { $match : { name : 'USAL' } },

  { $project : { _id : 0, name : 1 } },

  { $lookup : {

    from : 'courses',

    localField : 'name',

    foreignField : 'university',

    as : 'courses'

  } }

]).pretty()
```

MongoDB \$sortByCount

Esta etapa es un atajo para agrupar, contar y luego ordenar en orden descendente el número de valores diferentes de un campo.

Supongamos que desea conocer el número de cursos por nivel, ordenados de forma descendente. La siguiente es la consulta que tendría que construir:

```
db.courses.aggregate([

  { $sortByCount : '$level' }

]).pretty()
```

MongoDB \$facet

A veces, al crear un informe sobre datos, uno se encuentra con que necesita hacer el mismo tratamiento preliminar para varios informes, y se enfrenta a tener que crear y mantener una colección intermedia.

Puede, por ejemplo, hacer un resumen semanal de la negociación que sea utilizado por todos los informes posteriores. Tal vez haya deseado que fuera posible ejecutar más de un pipeline simultáneamente sobre la salida de un único pipeline de agregación.

Ahora podemos hacerlo dentro de una única canalización gracias a la función `$facet` etapa.